

Basics

THE GRAMMAR OF FLEXPLOT

flexplot builds plots from three components: a **formula**, a **dataset**, and automatic **geom** selection. The formula determines what goes where; flexplot picks the right visual marks.

outcome ~ predictor + color | panel_col + panel_row

Component	Role
outcome	Y-axis variable
predictor	X-axis variable
color	2nd predictor → colors/symbols
	Separator for paneling
panel_col	1st given → column facets
panel_row	2nd given → row facets

Max 2 given variables (row + column). 3+ → error.

COMPLETE TEMPLATE

```
flexplot(
  outcome ~ predictor + color
  | panel_col + panel_row,
  data = dataframe,
  method = "auto",      # "lm" | "loess"
  se = TRUE,            # CI band
  ghost.line = NULL,    # "black" for ref
  bins = NULL,          # bin numeric given
  spread = NULL,        # "quartiles"
  jitter = c(.1, 0)     # width/height
)
```

required: formula, data

optional: method, se, ghost.line, bins, spread, jitter

QUICK EXAMPLES

```
flexplot(satisfaction ~ communication
  + separated | gender
  + interests,
  data = relationship_satisfaction)
```

```
# Histogram (intercept-only)
flexplot(satisfaction ~ 1, data = df)
```

```
# Bar chart (categorical)
flexplot(gender ~ 1, data = df)
```

```
# Ghost line cross-panel
flexplot(y ~ x | group,
  data = df,
  ghost.line = "black")
```

SAVING PLOTS

R Call	Action
ggsave("plot.png")	Save last plot
print(p)	Print stored plot
p + theme_bw()	Add ggplot layers

flexplot returns ggplot objects — extend with + .

PYTHON PORT

Python py-flexplot uses plotnine + statsmodels .

```
from pyflexplot import flexplot
import statsmodels.formula.api as smf
```

```
mod = smf.ols("y ~ x1", data=df).fit()
visualize(mod)
```

Automatic Geom Selection

ONE VARIABLE

Formula	Plot Type
y ~ 1 (numeric)	Histogram
y ~ 1 (factor)	Bar chart

```
flexplot(hwy ~ 1, data = mpg)
```

TWO VARIABLES — BOTH CONTINUOUS

Formula	Plot Type
y ~ x	Scatterplot + smooth
y ~ x + color	Scatter w/ colored lines

```
flexplot(hwy ~ cty, data = mpg,
  method = "lm")
flexplot(hwy ~ cty + drv, data = mpg)
```

method options:

Value	Smoothing
"auto"	LM (default)
"lm"	Linear regression
"loess"	Local polynomial

Binary 0/1 outcome → binomial GLM auto-selected.

TWO VARIABLES — MIXED TYPES

Y type	X type	Plot
Numeric	Categorical	Jitter + mean CI
Categorical	Numeric	GLM smooth
Categorical	Categorical	Jittered scatter

```
flexplot(satisfaction ~ gender,
  data = df,
  spread = "quartiles",
  jitter = c(.1, 0))
```

PANELING (FACETING)

Given Vars	Layout
1	facet_wrap(~v)
2	facet_grid(row~col)
3+	Error (max 2)

```
flexplot(y ~ x | group, data = df)
flexplot(y ~ x | g1 + g2, data = df)
```

Use bins = 3 to discretize numeric given vars.

GHOST LINES

ghost.line = "black" overlays a reference line from the full model across all panels.

```
flexplot(y ~ x | group,
  data = df,
  ghost.line = "black",
  se = FALSE)
```

Advanced Visualization

ADDED VARIABLE PLOTS

Partial regression plot — residualizes out other predictors.

```
added.plot(satisfaction
  ~ communication + separated
  | gender, data = df,
  method = "polynomial",
  se = FALSE)
```

Use for linearity checks, outlier detection, multicollinearity diagnosis.

VISUALIZING FITTED MODELS

Pass a fitted model to visualize() .

```
mod = lm(satisfaction
  ~ communication * separated,
  data = df)
visualize(mod)
```

Supports: lm , glm , any model with predict() . Produces predicted-vs-actual and diagnostic plots.

COMPARING MODEL FITS

Overlay two models visually.

```
full = lm(y ~ x * z, data = df)
reduced = lm(y ~ x + z, data = df)
compare.fits(y ~ x | z,
  data = df, full, reduced)
```

Blue = model 1, Red = model 2.

SEM VISUALIZATION

R flexplavaan port for semopy .

Function	Visualizes
hopper_plot(mod)	Covariance residuals
disturbance_plot(...)	Latent disturbance
measurement_plot(...)	Latent→indicator

```
mod = Model("F1 =~ x1 + x2")
mod.fit(data)
hopper_plot(mod)
```

NEURAL NETWORKS (FLEX_NN)

Drop torch.nn.Module or keras.Model into the pipeline.

```
fit = NeuralNetFit(model,
  X_train, y_train,
  predictor_names=["x1"],
  response_var="y")
visualize(fit, data=test_df)
permutation_importance(fit)
```

Backends: torch (default), Keras 3.

FORMULA SYNTAX REFERENCE

Formula	Meaning
y ~ 1	Univariate dist
y ~ x	Bivariate scatter
y ~ x + z	x axis, z = color
y ~ x g	Panel by g
y ~ x g1+g2	g1 cols, g2 rows
y ~ x+z g1+g2	Full model

Companion Packages

FIFER / FIFER2 — BIOSTATISTICS

Function	Purpose
model_comparison(m1, m2)	Compare two models
estimates(mod)	Extract coefficients
p_format(p, digits=3)	Format p-values
eliminated_columns(df, .5)	Flag near-constant cols
color_table(df, "viridis")	Correlation heatmap

```
m1 = lm(y ~ x1, data = df)
m2 = lm(y ~ x1 + x2, data = df)
model_comparison(m1, m2)
```

EBBR — EMPIRICAL BAYES

Beta-Binomial shrinkage estimation.

Function	Purpose
fit_beta_prior(s, n)	Fit Beta prior
add_ebb_estimate(df, s, n)	Add shrinkage est.

```
prior = fit_beta_prior(
  successes, totals)
df = add_ebb_estimate(df,
  "hits", "at_bats")
```

BLUEPILL — SYNTHETIC DATA

Generate clustered data with fixed + random effects.

Function	Purpose
estimate_sd(mean, min, max)	Recover SD from range
mixed_model(...)	Generate mixed-model data

```
df = mixed_model(
  cluster = c(school=10, student=5),
  fixed = list(
    math = c(mean=55, sd=10, n=100),
    ses = c(mean=0, sd=1, n=100)),
  random = list(
    school = list(intercept=c(0,5)),
    student = list(intercept=c(0,3))),
  errors = c(0, 5),
  treatment = list(
    levels = c("ctrl","tx"),
    prop = c(.5,.5)),
  interactions = list(
    c("math","ses", coef=0.3)),
  polynomials = list(
    math = c(from=1, to=2, coef=0.1)))
```

PACKAGE MAP

Package	Module
flexplot	core.py
fifer/fifer2	stats.py
flexplavaan	sem.py
flex_nn	flex_nn.py
bluepill	bluepill.py
ebbr	ebbr.py

INSTALL

```
# R
devtools::install_github(
  "dustinfife/flexplot")
library(flexplot)
```

```
# Python
pip install git+https://github.com/
ezraair555/py-flexplot.git
from pyflexplot import *
```